



University
of Exeter

COMM113

Deep Learning – Week 5

Convolutional Neural Networks

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Pooling Layer

- Spatial dimension reduction
- Local invariance
- No parameters: max or average of 2x2 (or nxn) units

What's the main benefit of pooling?

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

max pool with 2x2 filters
and stride 2



6	8
3	4

$$(4-2)/2 + 1 = 2$$

$$(\text{Input size} - \text{filter size})/\text{stride} + 1$$

Pooling Layer

- Spatial dimension reduction
- Enforce local invariance
- No parameters: max or average of 2x2 (or nxn) units

What's the main benefit of pooling?

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

max pool with 2x2 filters
and stride 2



6	8
3	4

What's the output size if input = 4 and filter = 2,
stride = 2?

$$(4-2)/2 + 1 = 2$$

$$(\text{Input size} - \text{filter size})/\text{stride} + 1$$

Pooling Layer

- Spatial dimension reduction
- Enforce local invariance
- No parameters: max or average of 2×2 (or $n \times n$) units

What's the main benefit of pooling?

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

max pool with 2×2 filters
and stride 2

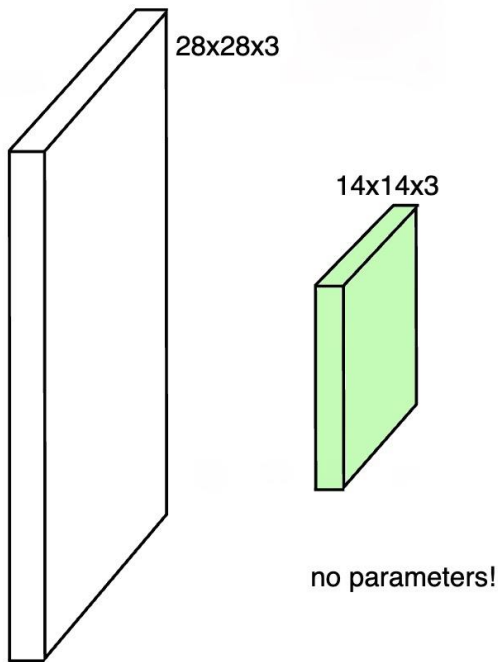


6	8
3	4

What's the output size if input = 4 and filter = 2, stride = 2?

Pooling Layer

- Spatial dimension reduction
- Local invariance
- No parameters: max or average of 2×2 (or $n \times n$) units



Convolution Layer

- Input size ($7 \times 7 \times 3$)
- Filter size ($5 \times 5 \times 3$)
- Stride = 2, No padding

Output size?

Convolution Layer

- Input size (7*7*3)
- Filter size (5*5*3)
- Stride = 2, No padding

Output size?

$$(7-5)/2 + 1 = 2$$

Classic ConvNet Architecture

Input

Conv blocks

- Convolution + activation (relu)
- Convolution + activation (relu)
- ...
- Maxpooling 2x2

Output

- Fully connected layers
- Softmax

Where do most parameters reside —
convolutional or fully connected layers?

Classic ConvNet Architecture

Input

Conv blocks

- Convolution + activation (relu)
- Convolution + activation (relu)
- ...
- Maxpooling 2x2

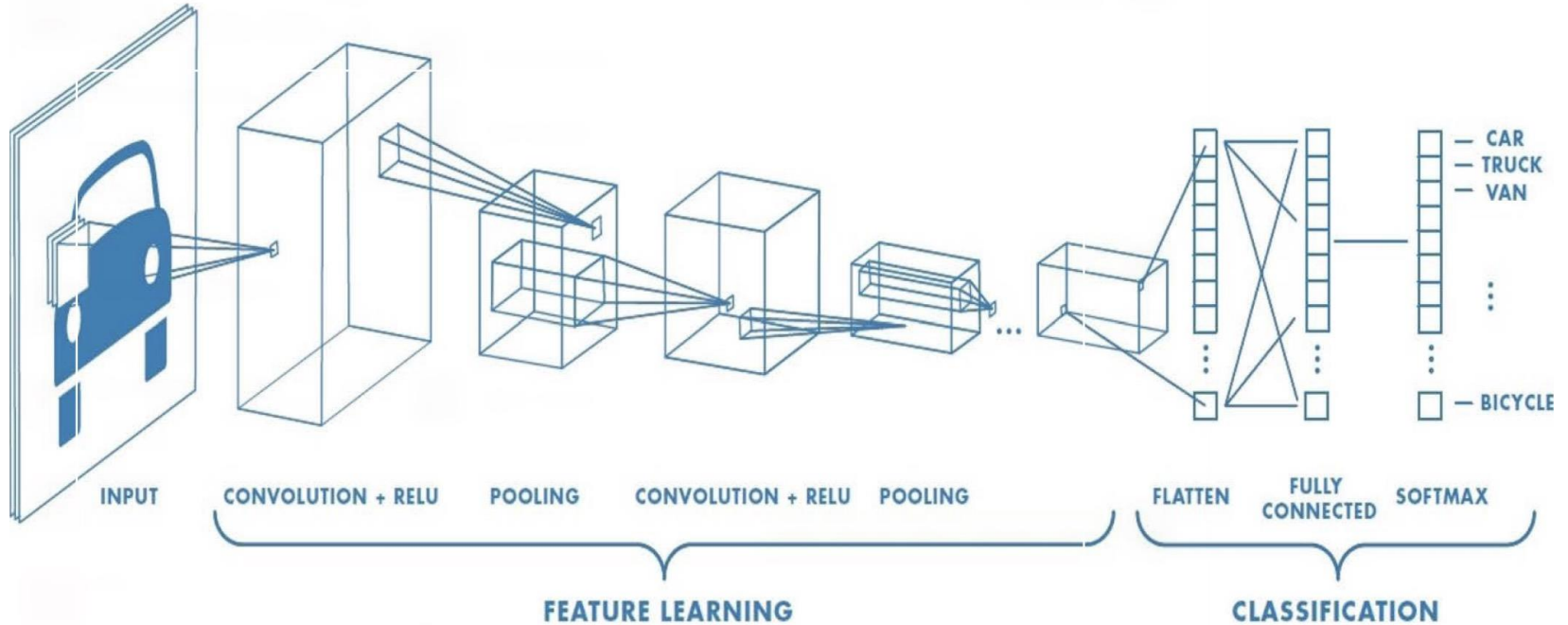
Output

- Fully connected layers
- Softmax

Where do most parameters reside —
convolutional or fully connected layers?

Usually in the fully connected layers.

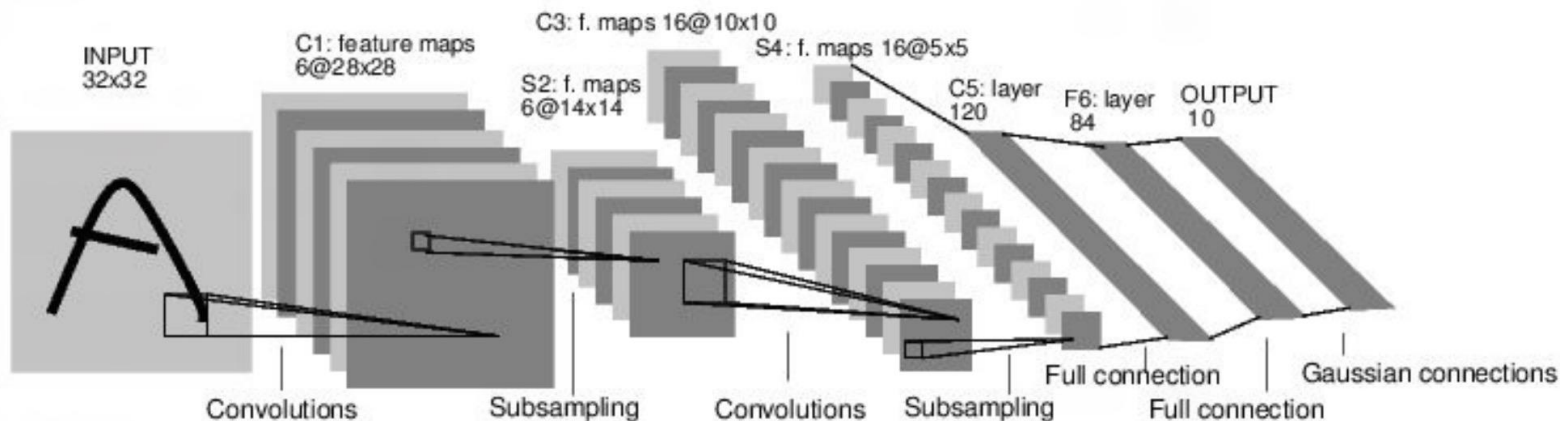
Convolution Neural Network



LeNet-5

Why were small filters (5×5) sufficient here?

[LeCun et al., 1998]



LeNet-5 was the first successful CNN, used for digit recognition on MNIST.

Conv filters were 5x5, applied at stride 1

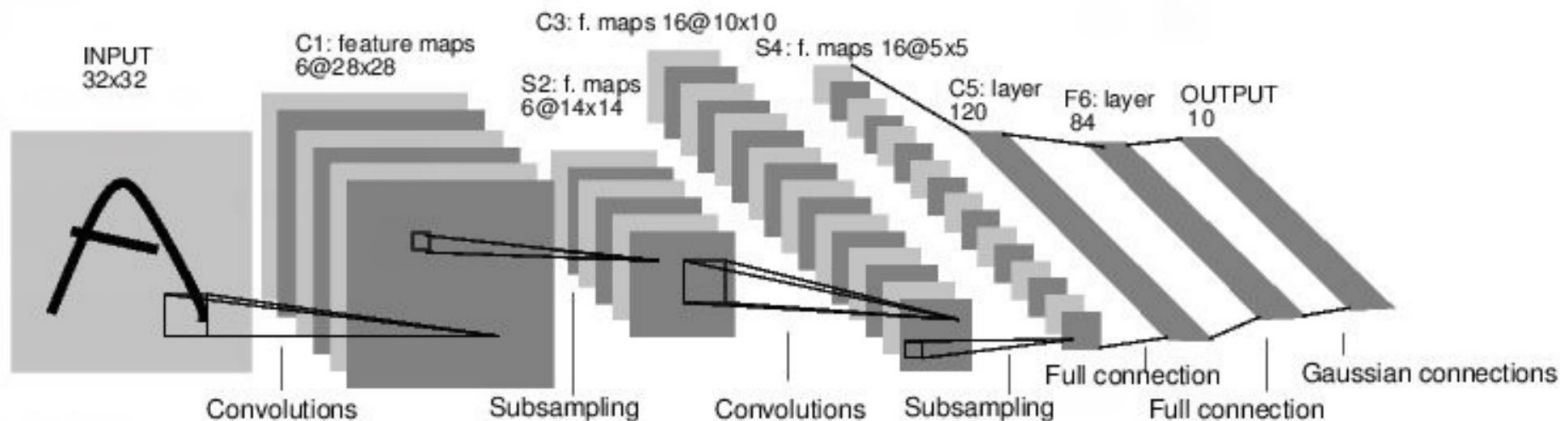
Pooling layers were 2x2 applied at stride 2

i.e. architecture is [CONV-POOL-CONV-POOL-CONV-FC]

LeNet-5

Why were small filters (5×5) sufficient here?
Digits are simple; fewer details to capture.

[LeCun et al., 1998]



LeNet-5 was the first successful CNN, used for digit recognition on MNIST.

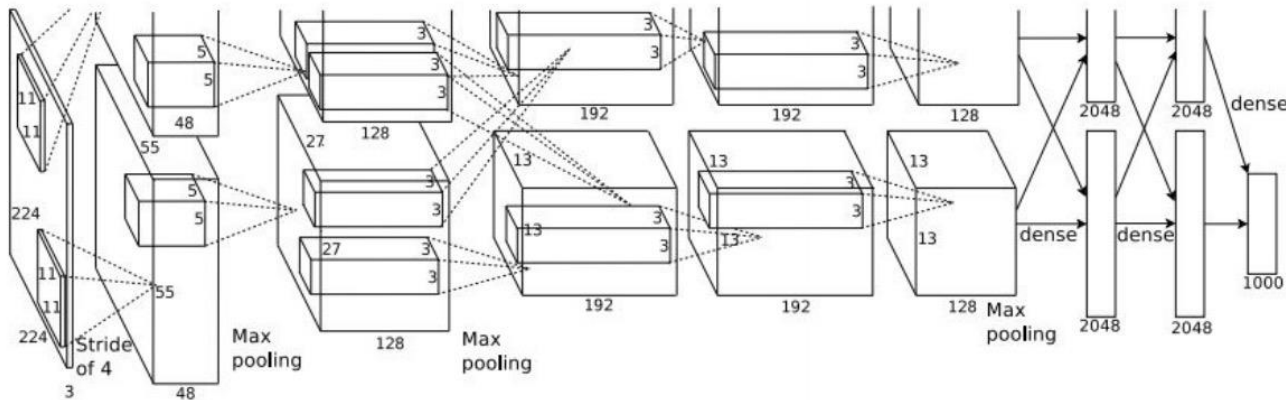
Conv filters were 5×5 , applied at stride 1

Pooling layers were 2×2 applied at stride 2

i.e. architecture is [CONV-POOL-CONV-POOL-CONV-FC]

[Krizhevsky et al. 2012]

ReLU activations.



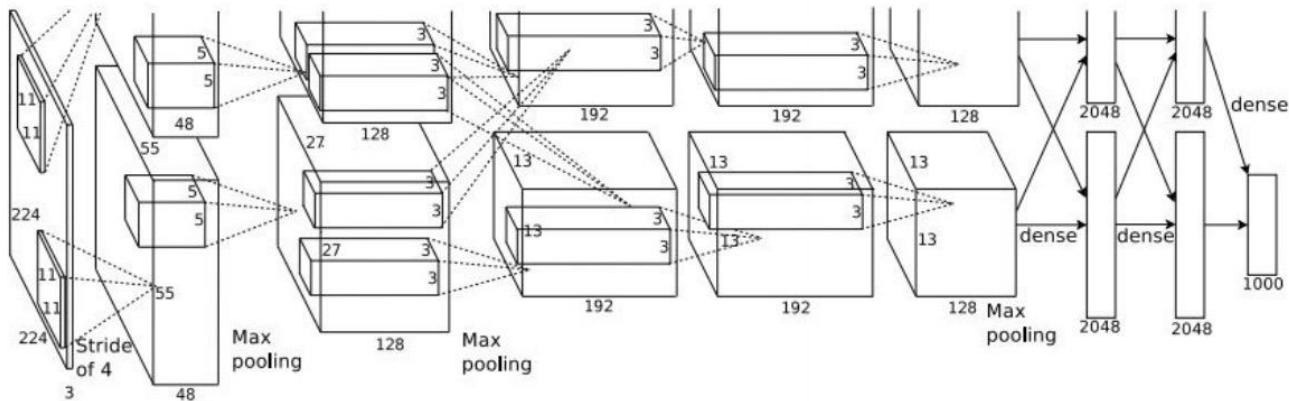
Input: 227x227x3 images

First layer (CONV1): 96 11x11x3 filters applied at stride 4

Q: what is the output volume size?

AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

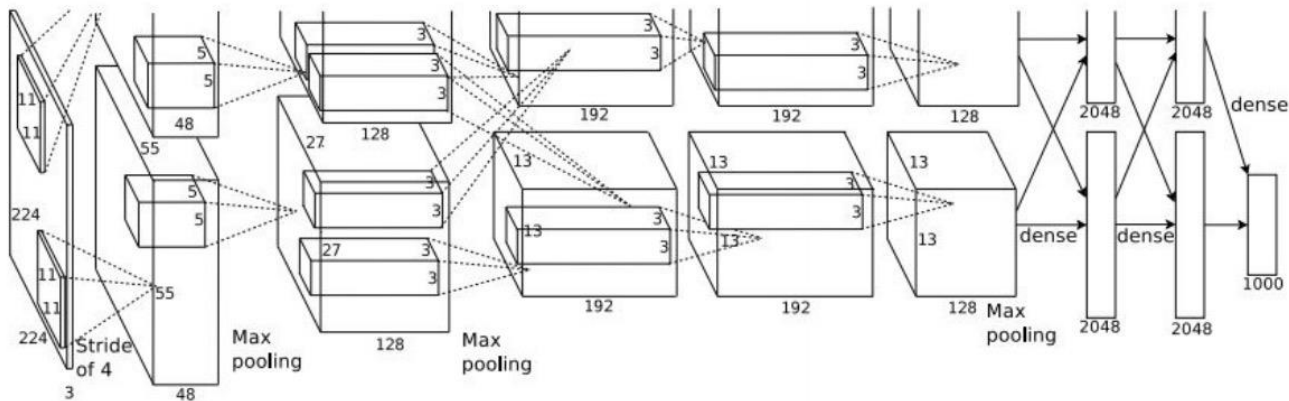
First layer (CONV1): 96 11x11x3 filters applied at stride 4

Q: what is the output volume size?

$$(227-11)/4+1 = 55$$

AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

First layer (CONV1): 96 11x11x3 filters applied at stride 4

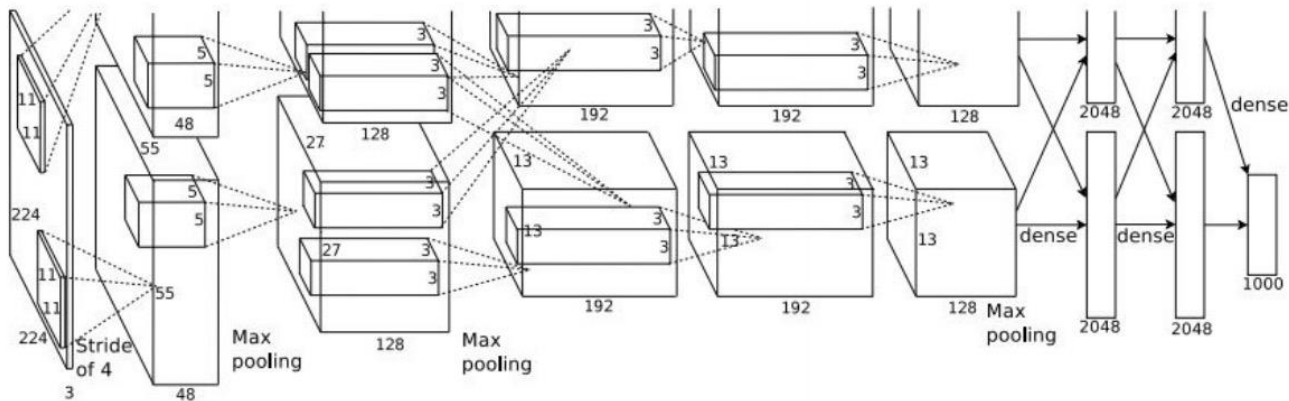
=>

Output volume [55x55x96]

Q: What is the total number of parameters in this layer?

AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

First layer (CONV1): 96 11x11x3 filters applied at stride 4

=>

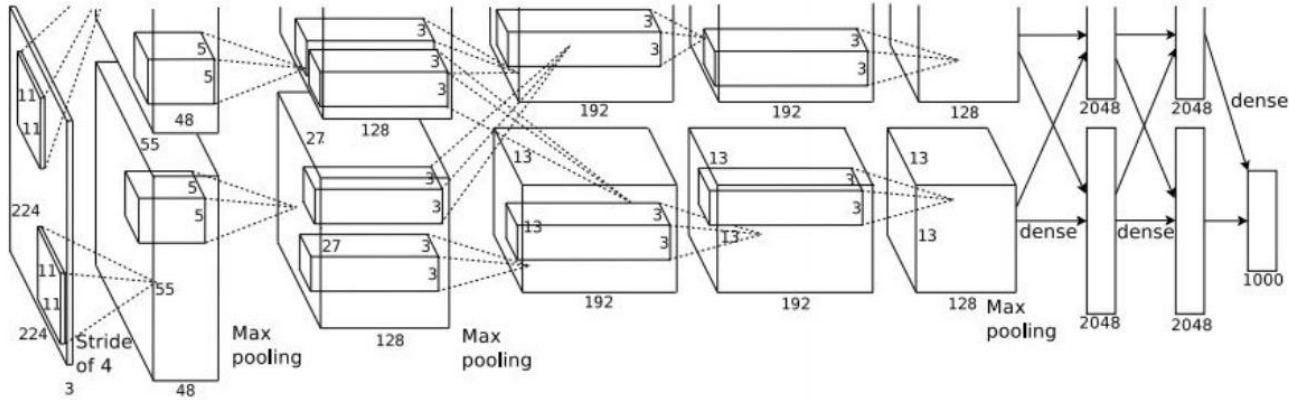
Output volume [55x55x96]

Q: What is the total number of parameters in this layer?

$(11*11*3)*96 = 35K$

AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

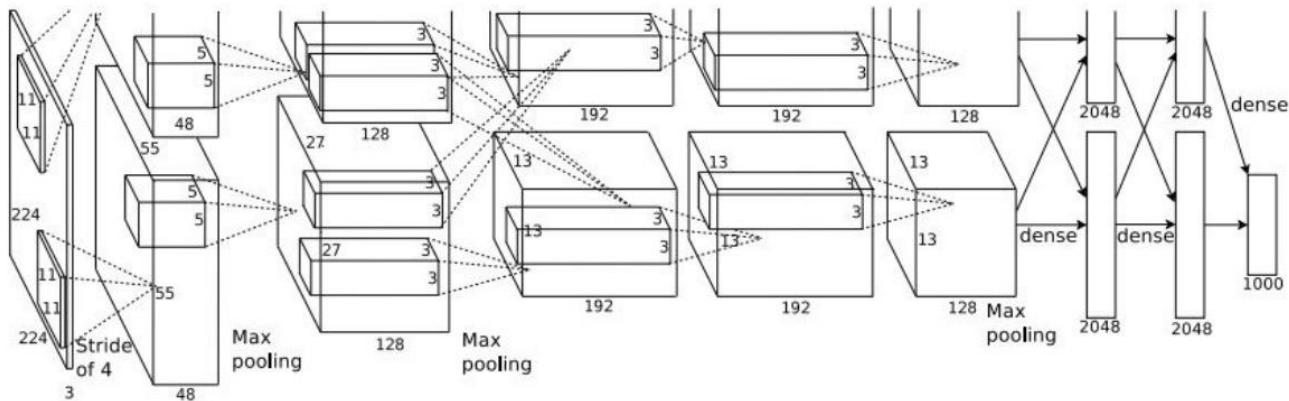
After CONV1: 55x55x96

Second layer (POOL1): 3x3 filters applied at stride 2

Q: what is the output volume size?

AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

After CONV1: 55x55x96

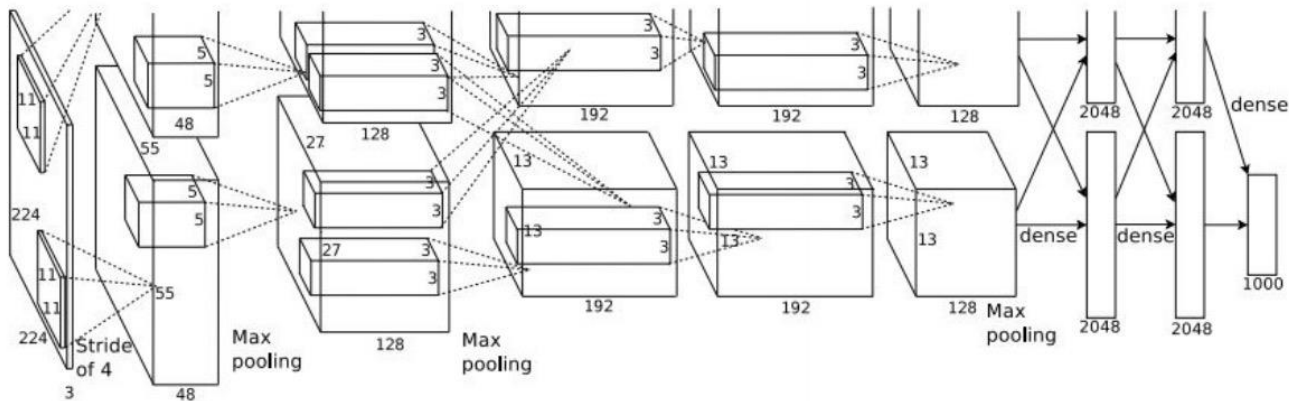
Second layer (POOL1): 3x3 filters applied at stride 2

Q: what is the output volume size?

$$(55-3)/2+1 = 27$$

AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

After CONV1: 55x55x96

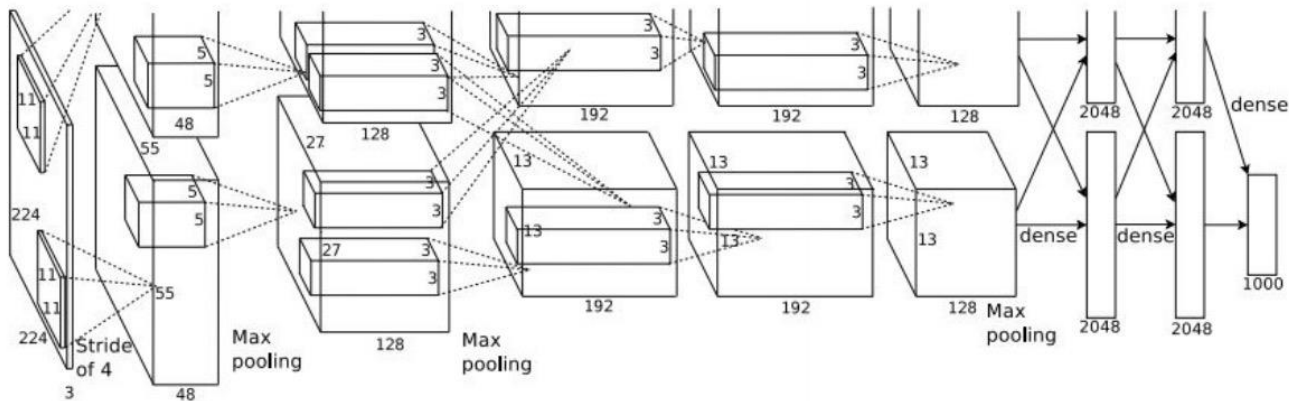
Second layer (POOL1): 3x3 filters applied at stride 2

Output volume: 27x27x96

Q: what is the number of parameters in this layer?

AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 images

After CONV1: 55x55x96

Second layer (POOL1): 3x3 filters applied at stride 2

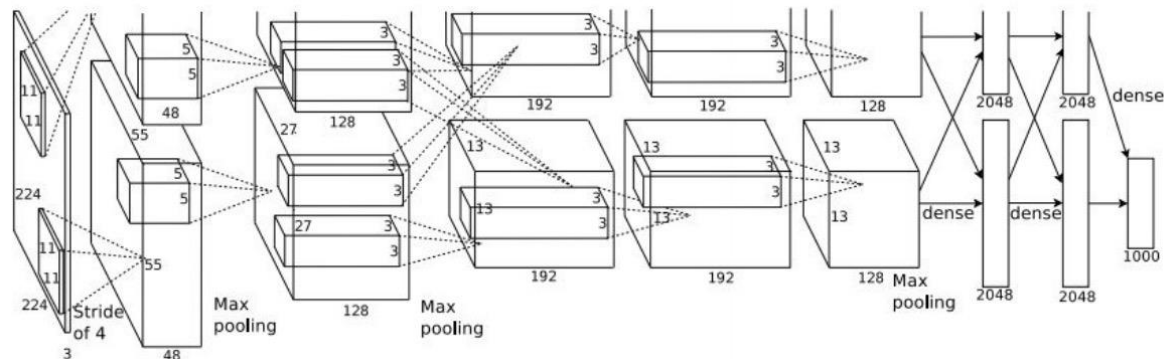
Output volume: 27x27x96

Q: what is the number of parameters in this layer?

No

AlexNet

[Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

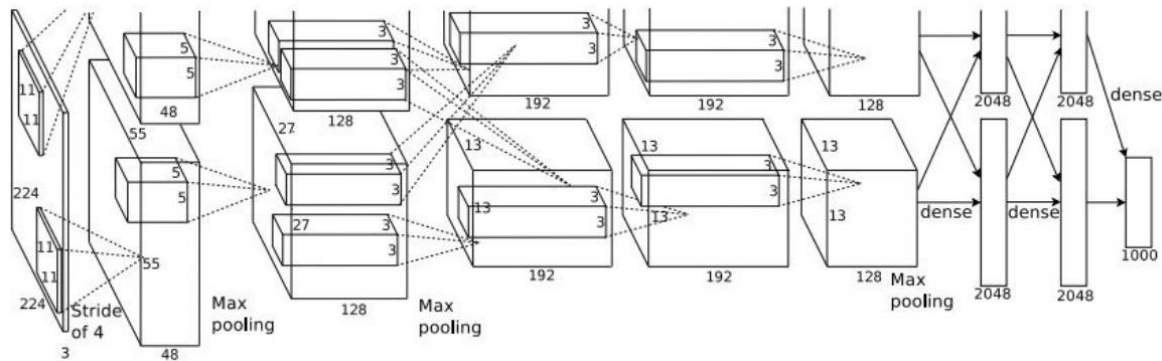
[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

Why use stride 4 in the first layer?

AlexNet

[Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

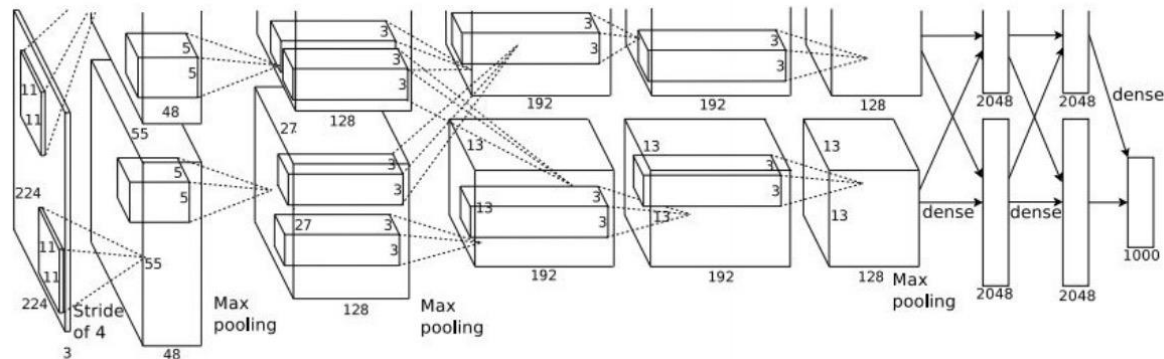
Why use stride 4 in the first layer?

To aggressively reduce resolution for large inputs.

Which layer dominates parameter count?

AlexNet

[Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

Why use stride 4 in the first layer?

To aggressively reduce resolution for large inputs.

Which layer dominates parameter count?

The fully connected layers at the end.

VGGNet

VGGNet introduced a simple yet powerful idea: use only 3×3 filters (stride 1, pad 1) and 2×2 pooling (stride 2).

Why prefer several 3×3 convs instead of one 7×7 conv?

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as “conv<receptive field size>-<number of channels>”. The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Best one

VGGNet

VGGNet introduced a simple yet powerful idea: use only 3×3 filters (stride 1, pad 1) and 2×2 pooling (stride 2).

Why prefer several 3×3 convs instead of one 7×7 conv?

Same receptive field, fewer parameters, more nonlinearities.

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as “conv<receptive field size>-<number of channels>”. The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Best one

VGGNet

INPUT: [224x224x3] **memory: 224*224*3=150K** params: 0

CONV3-64: [224x224x64] **memory: 224*224*64=3.2M** params: $(3*3*3)*64 = 1,728$

CONV3-64: [224x224x64] **memory: 224*224*64=3.2M** params: $(3*3*64)*64 = 36,864$

POOL2: [112x112x64] **memory: 112*112*64=800K** params: 0

CONV3-128: [112x112x128] **memory: 112*112*128=1.6M** params: $(3*3*64)*128 = 73,728$

CONV3-128: [112x112x128] **memory: 112*112*128=1.6M** params: $(3*3*128)*128 = 147,456$

POOL2: [56x56x128] **memory: 56*56*128=400K** params: 0

CONV3-256: [56x56x256] **memory: 56*56*256=800K** params: $(3*3*128)*256 = 294,912$

CONV3-256: [56x56x256] **memory: 56*56*256=800K** params: $(3*3*256)*256 = 589,824$

CONV3-256: [56x56x256] **memory: 56*56*256=800K** params: $(3*3*256)*256 = 589,824$

POOL2: [28x28x256] **memory: 28*28*256=200K** params: 0

CONV3-512: [28x28x512] **memory: 28*28*512=400K** params: $(3*3*256)*512 = 1,179,648$

CONV3-512: [28x28x512] **memory: 28*28*512=400K** params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [28x28x512] **memory: 28*28*512=400K** params: $(3*3*512)*512 = 2,359,296$

POOL2: [14x14x512] **memory: 14*14*512=100K** params: 0

CONV3-512: [14x14x512] **memory: 14*14*512=100K** params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] **memory: 14*14*512=100K** params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] **memory: 14*14*512=100K** params: $(3*3*512)*512 = 2,359,296$

POOL2: [7x7x512] **memory: 7*7*512=25K** params: 0

FC: [1x1x4096] **memory: 4096** params: $7*7*512*4096 = 102,760,448$

FC: [1x1x4096] **memory: 4096** params: $4096*4096 = 16,777,216$

FC: [1x1x1000] **memory: 1000** params: $4096*1000 = 4,096,000$

TOTAL memory: 24M * 4 bytes ~ = 93MB / image

TOTAL params: 138M parameters

Table 1: **ConvNet configurations** (shown in columns). The depth of the configuration from the left (A) to the right (E), as more layers are added (the added layers are shown in red). The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGGNet

INPUT: [224x224x3] **memory: $224*224*3=150K$** params: 0
CONV3-64: [224x224x64] **memory: $224*224*64=3.2M$** params: $(3*3*3)*64 = 1,728$
CONV3-64: [224x224x64] **memory: $224*224*64=3.2M$** params: $(3*3*64)*64 = 36,864$
POOL2: [112x112x64] **memory: $112*112*64=800K$** params: 0
CONV3-128: [112x112x128] **memory: $112*112*128=1.6M$** params: $(3*3*64)*128 = 73,728$
CONV3-128: [112x112x128] **memory: $112*112*128=1.6M$** params: $(3*3*128)*128 = 147,456$
POOL2: [56x56x128] **memory: $56*56*128=400K$** params: 0
CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*128)*256 = 294,912$
CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*256)*256 = 589,824$
CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*256)*256 = 589,824$
POOL2: [28x28x256] **memory: $28*28*256=200K$** params: 0
CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*256)*512 = 1,179,648$
CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*512)*512 = 2,359,296$
POOL2: [14x14x512] **memory: $14*14*512=100K$** params: 0
CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$
POOL2: [7x7x512] **memory: $7*7*512=25K$** params: 0
FC: [1x1x4096] **memory: 4096** params: $7*7*512*4096 = 102,760,448$
FC: [1x1x4096] **memory: 4096** params: $4096*4096 = 16,777,216$
FC: [1x1x1000] **memory: 1000** params: $4096*1000 = 4,096,000$

TOTAL memory: 24M * 4 bytes ~= 93MB / image

TOTAL params: 138M parameters

VGGNet

INPUT: [224x224x3] **memory: $224*224*3=150K$** params: 0

CONV3-64: [224x224x64] **memory: $224*224*64=3.2M$** params: $(3*3*3)*64 = 1,728$

CONV3-64: [224x224x64] **memory: $224*224*64=3.2M$** params: $(3*3*64)*64 = 36,864$

POOL2: [112x112x64] **memory: $112*112*64=800K$** params: 0

CONV3-128: [112x112x128] **memory: $112*112*128=1.6M$** params: $(3*3*64)*128 = 73,728$

CONV3-128: [112x112x128] **memory: $112*112*128=1.6M$** params: $(3*3*128)*128 = 147,456$

POOL2: [56x56x128] **memory: $56*56*128=400K$** params: 0

CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*128)*256 = 294,912$

CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*256)*256 = 589,824$

CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*256)*256 = 589,824$

POOL2: [28x28x256] **memory: $28*28*256=200K$** params: 0

CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*256)*512 = 1,179,648$

CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*512)*512 = 2,359,296$

POOL2: [14x14x512] **memory: $14*14*512=100K$** params: 0

CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$

POOL2: [7x7x512] **memory: $7*7*512=25K$** params: 0

FC: [1x1x4096] **memory: 4096** params: $7*7*512*4096 = 102,760,448$

FC: [1x1x4096] **memory: 4096** params: $4096*4096 = 16,777,216$

FC: [1x1x1000] **memory: 1000** params: $4096*1000 = 4,096,000$

Most memory is in early CONV

Most params are in late FC

Why does VGG require ~138 M parameters?

TOTAL memory: 24M * 4 bytes ~= 93MB / image

TOTAL params: 138M parameters

VGGNet

INPUT: [224x224x3] **memory: $224*224*3=150K$** params: 0
CONV3-64: [224x224x64] **memory: $224*224*64=3.2M$** params: $(3*3*3)*64 = 1,728$
CONV3-64: [224x224x64] **memory: $224*224*64=3.2M$** params: $(3*3*64)*64 = 36,864$
POOL2: [112x112x64] **memory: $112*112*64=800K$** params: 0
CONV3-128: [112x112x128] **memory: $112*112*128=1.6M$** params: $(3*3*64)*128 = 73,728$
CONV3-128: [112x112x128] **memory: $112*112*128=1.6M$** params: $(3*3*128)*128 = 147,456$
POOL2: [56x56x128] **memory: $56*56*128=400K$** params: 0
CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*128)*256 = 294,912$
CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*256)*256 = 589,824$
CONV3-256: [56x56x256] **memory: $56*56*256=800K$** params: $(3*3*256)*256 = 589,824$
POOL2: [28x28x256] **memory: $28*28*256=200K$** params: 0
CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*256)*512 = 1,179,648$
CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [28x28x512] **memory: $28*28*512=400K$** params: $(3*3*512)*512 = 2,359,296$
POOL2: [14x14x512] **memory: $14*14*512=100K$** params: 0
CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$
CONV3-512: [14x14x512] **memory: $14*14*512=100K$** params: $(3*3*512)*512 = 2,359,296$
POOL2: [7x7x512] **memory: $7*7*512=25K$** params: 0
FC: [1x1x4096] **memory: 4096** params: $7*7*512*4096 = 102,760,448$
FC: [1x1x4096] **memory: 4096** params: $4096*4096 = 16,777,216$
FC: [1x1x1000] **memory: 1000** params: $4096*1000 = 4,096,000$

Most memory is in early CONV

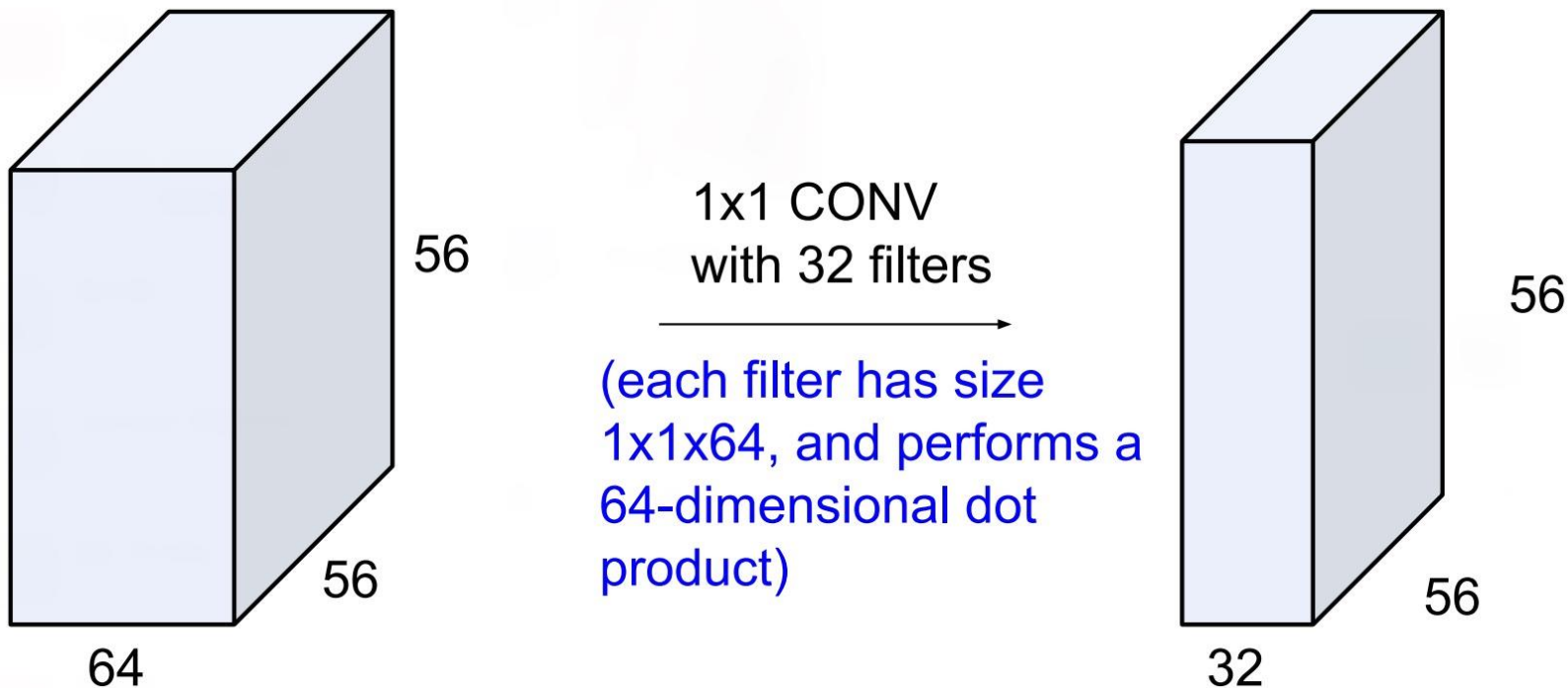
Most params are in late FC

TOTAL memory: $24M * 4 \text{ bytes} \approx 93MB$ / image
TOTAL params: 138M parameters

Why does VGG require ~138 M parameters?
Huge FC layers; each FC connects all neurons.

1x1 Convolutions

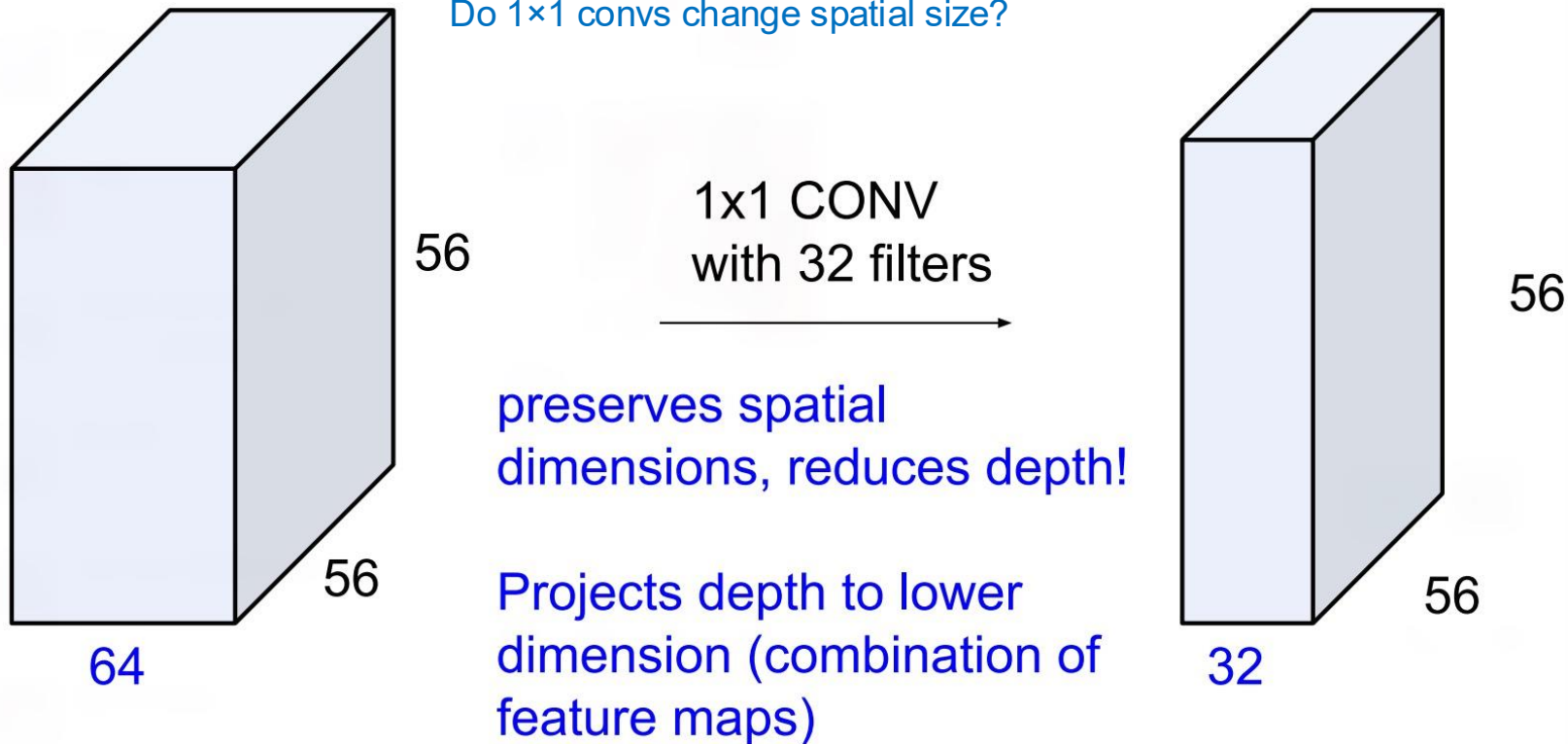
1×1 convs sound trivial, but they're crucial. They reduce channel depth and allow lightweight nonlinear combinations of feature maps (used heavily in GoogLeNet, ResNet, MobileNet).



1x1 Convolutions

If we apply 64 1×1 filters on a 128-channel feature map, what's the output depth?

Do 1×1 convs change spatial size?

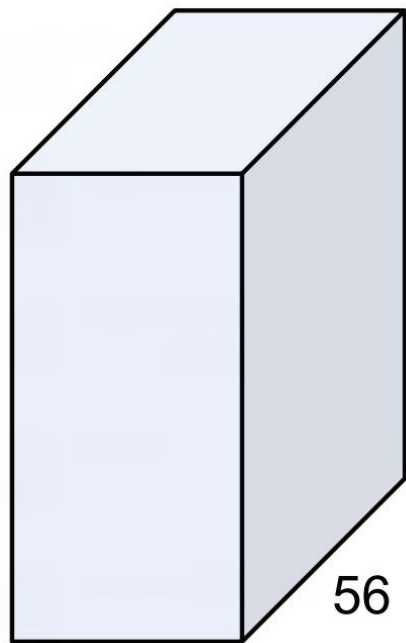


1x1 Convolutions

If we apply 64 1×1 filters on a 128-channel feature map, what's the output depth?
64.

Do 1×1 convs change spatial size?

No, only the depth.



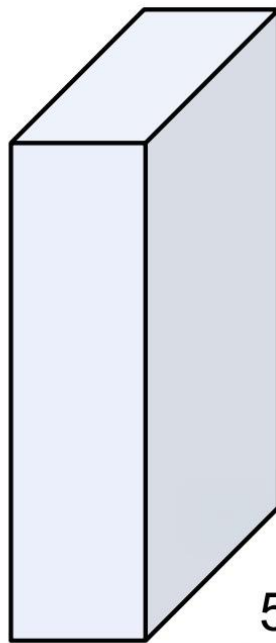
56

1x1 CONV
with 32 filters



preserves spatial
dimensions, reduces depth!

Projects depth to lower
dimension (combination of
feature maps)



56

56

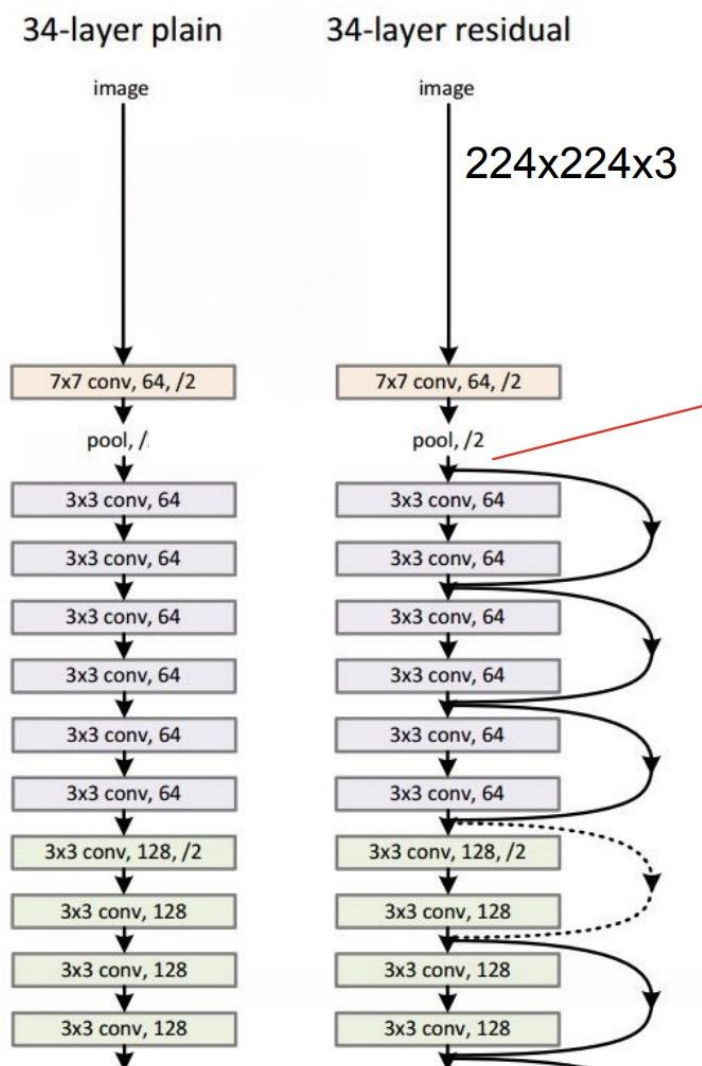
32

ResNet

[He et al., 2015]

As networks grew deeper, they became harder to optimise.
ResNet solved this with *residual connections*.

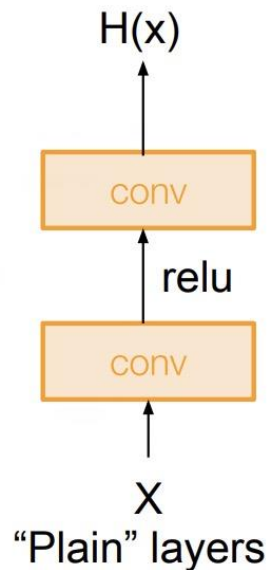
For deeper models, smaller or even 0 gradients in the earlier weights.



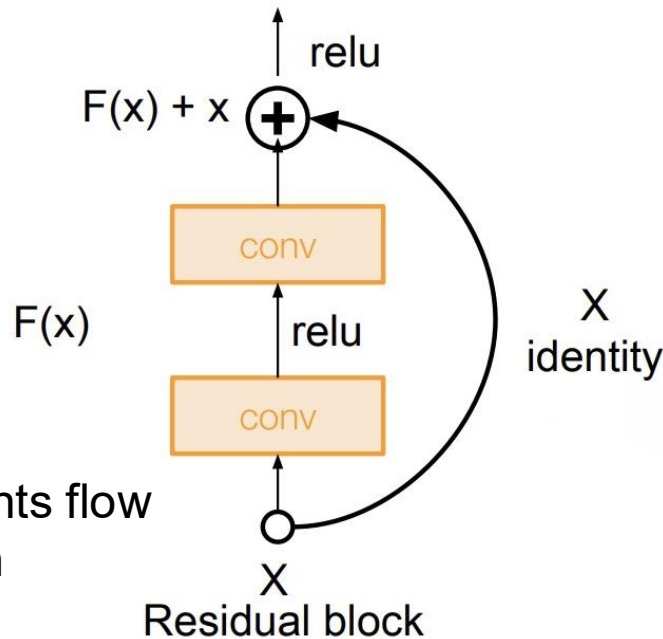
ResNet

[He et al., 2015]

Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



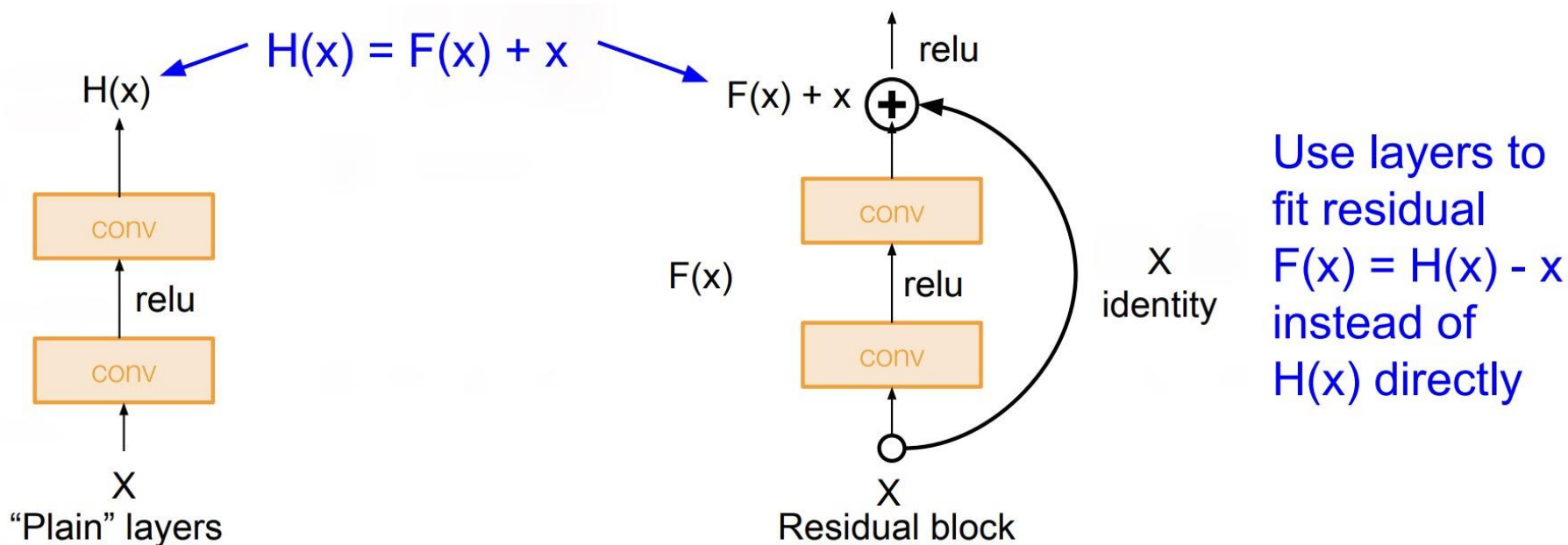
This helps gradients flow backward through identity paths.



ResNet

[He et al., 2015]

Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

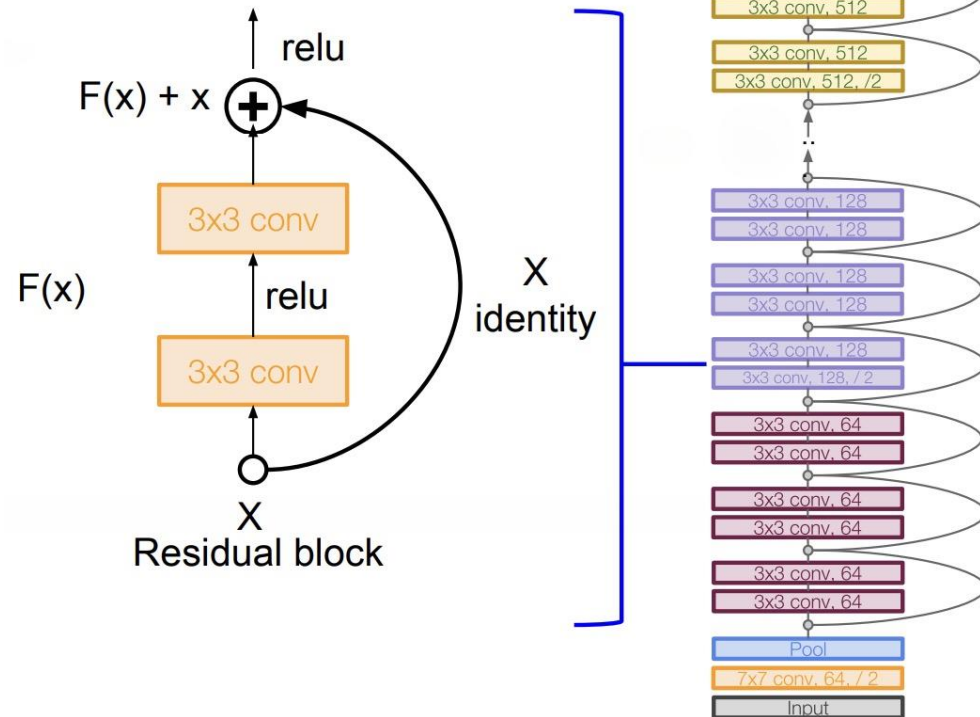


ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers

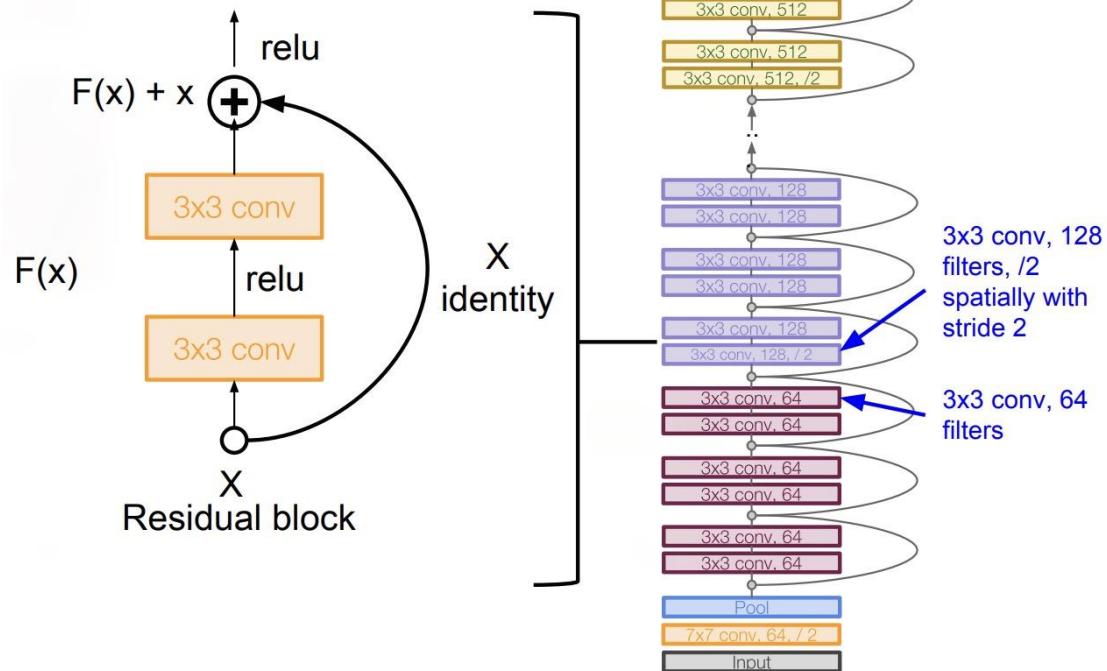


ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2

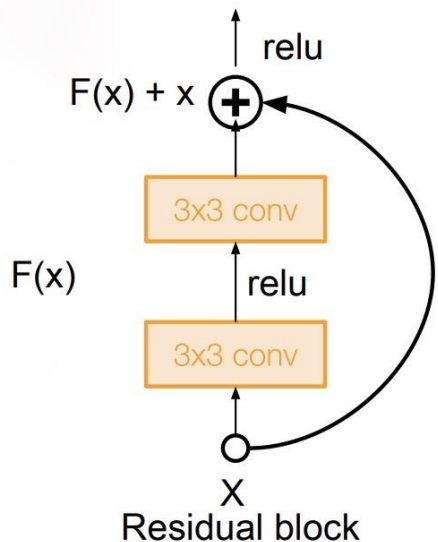


ResNet

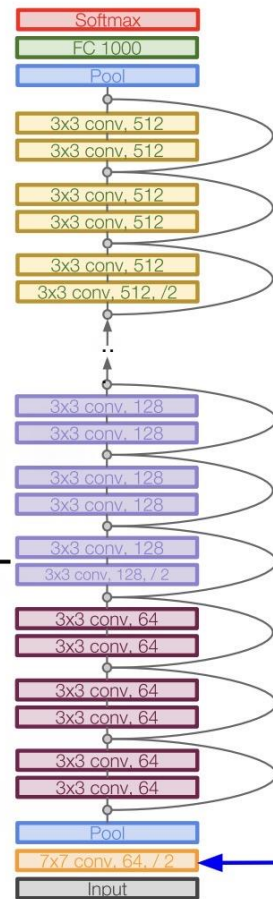
[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2
- Additional conv layer at the beginning



$\times$
identity



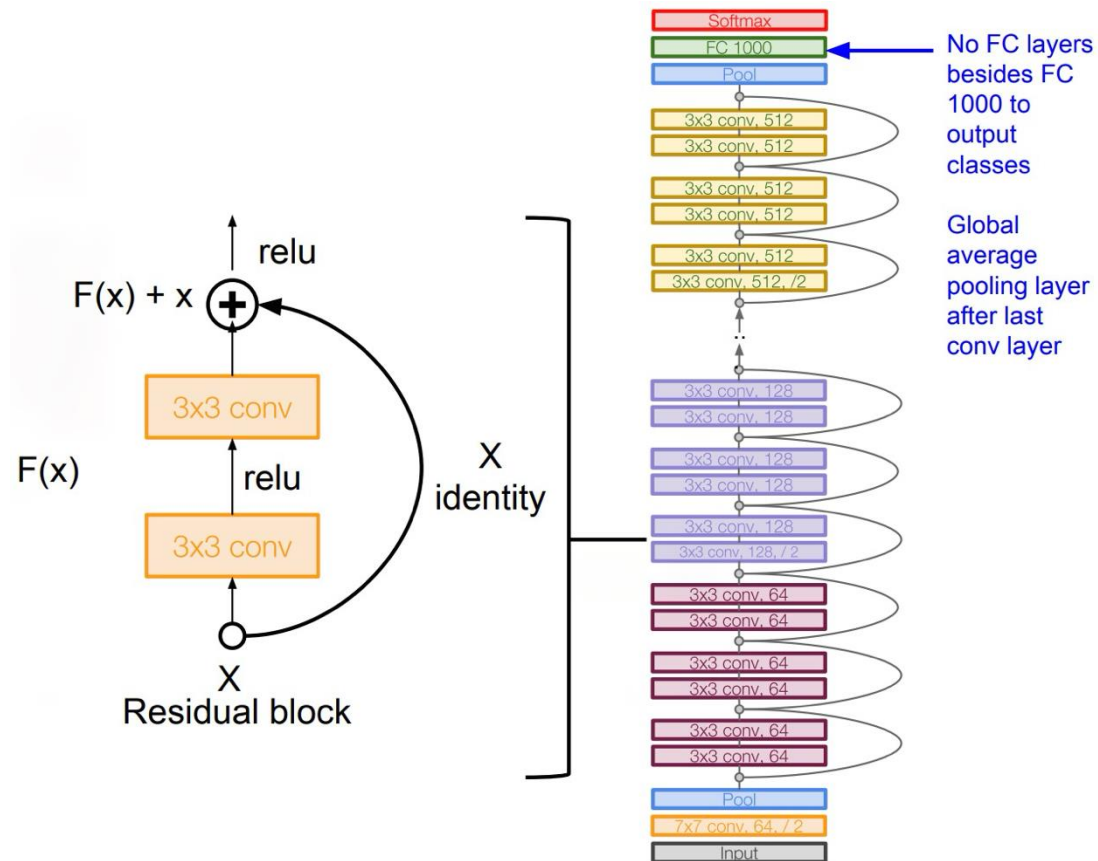
Beginning
conv layer

ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2
- Additional conv layer at the beginning
- No FC layers at the end (only FC 1000 to output classes)

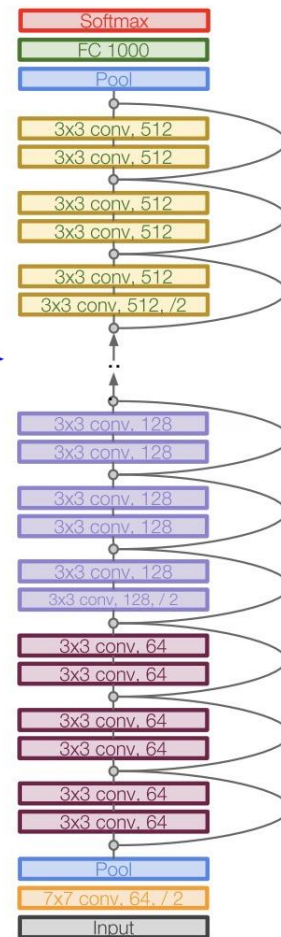


ResNet

[He et al., 2015]

Total depths of 34, 50, 101, or 152 layers for ImageNet

What problem do residual connections solve?



ResNet

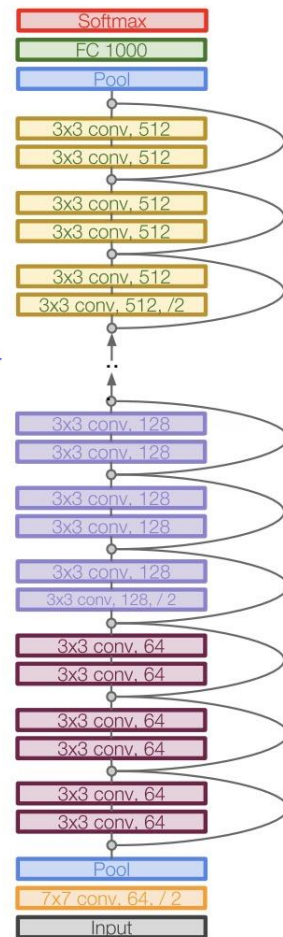
[He et al., 2015]

Total depths of 34, 50, 101, or
152 layers for ImageNet



What problem do residual connections solve?
Vanishing gradients in deep nets.

How do skip connections affect optimisation?



ResNet

[He et al., 2015]

Total depths of 34, 50, 101, or
152 layers for ImageNet

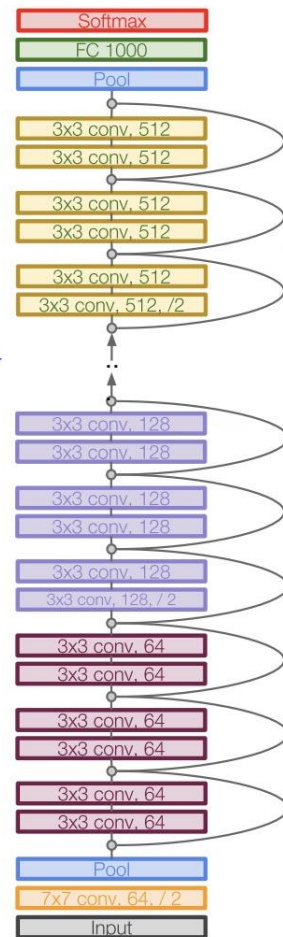


What problem do residual connections solve?

Vanishing gradients in deep nets.

How do skip connections affect optimisation?

They provide direct gradient flow, enabling
100+ layers.



Implement in PyTorch (CNN)

```
def __init__(self, num_classes=10):
    super(BasicCNN, self).__init__()

    # Convolutional layers
    self.conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
    self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)

    # Pooling layer
    self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

    # Fully connected layers
    # Assuming input image size is 32x32 (like CIFAR-10)
    # After 2 pooling layers: 32x32 -> 16x16 -> 8x8
    self.fc1 = nn.Linear(64 * 8 * 8, 512)
    self.fc2 = nn.Linear(512, 128)
    self.fc3 = nn.Linear(128, num_classes)

    # Dropout for regularization
    self.dropout = nn.Dropout(0.5)
```

```
def forward(self, x):
    # Conv layer 1 + ReLU + Pooling
    x = self.pool(F.relu(self.conv1(x)))

    # Conv layer 2 + ReLU + Pooling
    x = self.pool(F.relu(self.conv2(x)))

    # Flatten the feature maps
    x = x.view(-1, 64 * 8 * 8)

    # Fully connected layers
    x = F.relu(self.fc1(x))
    x = self.dropout(x)
    x = F.relu(self.fc2(x))
    x = self.dropout(x)
    x = self.fc3(x)

    return x
```

PyTorch (AlexNet)

```
# Classification layers
self.classifier = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(256 * 6 * 6, 4096),
    nn.ReLU(inplace=True),

    nn.Dropout(0.5),
    nn.Linear(4096, 4096),
    nn.ReLU(inplace=True),

    nn.Linear(4096, num_classes),
)

def forward(self, x):
    x = self.features(x)
    x = self.avgpool(x)
    x = torch.flatten(x, 1)
    x = self.classifier(x)
    return x
```



```
def __init__(self, num_classes=1000):
    super(AlexNet, self).__init__()

    # Feature extraction layers
    self.features = nn.Sequential(
        # Layer 1: Conv + ReLU + MaxPool
        nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=3, stride=2),

        # Layer 2: Conv + ReLU + MaxPool
        nn.Conv2d(64, 192, kernel_size=5, padding=2),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=3, stride=2),

        # Layer 3: Conv + ReLU
        nn.Conv2d(192, 384, kernel_size=3, padding=1),
        nn.ReLU(inplace=True),

        # Layer 4: Conv + ReLU
        nn.Conv2d(384, 256, kernel_size=3, padding=1),
        nn.ReLU(inplace=True),

        # Layer 5: Conv + ReLU + MaxPool
        nn.Conv2d(256, 256, kernel_size=3, padding=1),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=3, stride=2),
    )

    # Adaptive pooling to handle different input sizes
    self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
```

PyTorch (VGG)

```
def _make_layers(self, config, batch_norm=False):
    layers = []
    in_channels = 3

    for v in config:
        if v == 'M':
            layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
        else:
            conv2d = nn.Conv2d(in_channels, v, kernel_size=3, padding=1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(v), nn.ReLU(inplace=True)]
            else:
                layers += [conv2d, nn.ReLU(inplace=True)]
            in_channels = v

    return nn.Sequential(*layers)
```

Repeat



```
def __init__(self, vgg_config, num_classes=1000, batch_norm=False):
    super(VGG, self).__init__()
    self.features = self._make_layers(vgg_config, batch_norm)
    self.avgpool = nn.AdaptiveAvgPool2d((7, 7))
    self.classifier = nn.Sequential(
        nn.Linear(512 * 7 * 7, 4096),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(4096, 4096),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(4096, num_classes),
    )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

Summary

Convolution Layers: Learn spatially local filters → detect edges, shapes, textures

Stride & Padding: Control output resolution and preserve border information

Activation (ReLU): Adds nonlinearity and sparsity

Pooling Layers: Reduce spatial dimension, achieve translation invariance

Fully Connected Layers: Combine extracted features for classification

LeNet-5 (1998): First successful CNN for handwritten digits

AlexNet (2012): Deep CNN + ReLU + GPU training revolutionised deep learning

VGGNet (2014): Simplified design using only 3×3 convs → deeper, modular

ResNet (2015): Introduced residual connections → enabled ultra-deep networks

Q & A

No lectures and workshop next week (reading week)